02. SP1 프로젝트 구조 분석

개요

SP1 프로젝트는 Rust 워크스페이스 구조로 구성되어 있으며, 각각의 crate가 특정한 역할과 책임을 가지고 있습니다. 본 문서에서는 SP1 프로젝트의 전체 구조를 분석하고 각 crate의 역할을 상세히 설명합니다.

전체 프로젝트 구조

```
sp1/
├── crates/ # 핵심 라이브러리 crate들
| ├── build/ # 빌드 시스템
| ├── cli/ # 명령줄 인터페이스
| ├── core/ # 핵심 실행 및 머신 로직
| | ├── executor/ # RISC-V 실행 엔진
| | └── machine/ # 증명 생성 머신
| ├── cuda/ # CUDA 가속화
| ├── curves/ # 타원곡선 암호학
| ├── derive/ # 매크로 및 파생 구현
| ├── eval/ # 평가 및 벤치마킹
| ├── helper/ # 유틸리티 함수
| ├── perf/ # 성능 측정 도구
```

```
| ├── primitives/ # 기본 암호학 프리미티브
| ├── prover/ # 증명 생성 시스템
| ├── recursion/ # 재귀적 증명 시스템
| | ├── circuit/ # 회로 구현
| | ├── compiler/ # 컴파일러
| | ├── core/ # 재귀 핵심 로직
| | ├── derive/ # 재귀 매크로
| | ├── gnark-cli/ # Gnark CLI 인터페이스
| | └── gnark-ffi/ # Gnark FFI 바인딩
| ├── sdk/ # 개발자 SDK
| ├── stark/ # STARK 증명 시스템
| ├── test-artifacts/ # 테스트 아티팩트
| ├── verifier/ # 증명 검증기
| └── zkvm/ # zkVM 런타임
| ├── entrypoint/ # 프로그램 진입점
| └── lib/ # zkVM 라이브러리
├── examples/ # 예제 프로그램들
├── audits/ # 보안 감사 보고서
└── docs/ # 문서 (본 분석 문서 포함)
```

핵심 Crate 분석

1. SDK (sp1-sdk)

역할: 개발자가 SP1을 사용하기 위한 고수준 인터페이스 제공

주요 기능:

- ProverClient: 증명 생성 클라이언트 인터페이스
- CpuProver: CPU 기반 증명 생성
- CudaProver: GPU 가속 증명 생성
- NetworkProver: 네트워크 기반 증명 생성
- 증명 검증 기능

의존성:

- sp1-prover: 실제 증명 생성 로직
- sp1-core-machine: 머신 상태 관리
- sp1-core-executor: 프로그램 실행
- sp1-cuda: CUDA 가속화 (선택적)

2. Core Executor (sp1-core-executor)

역할: RISC-V 프로그램의 실행 및 실행 추적 생성

주요 기능:

- RISC-V 명령어 해석 및 실행
- 메모리 상태 추적
- 시스템 콜 처리
- 실행 통계 수집
- ELF 바이너리 로딩

핵심 구성요소:

- Executor: 메인 실행 엔진

- Program: 프로그램 표현
- Runtime: 런타임 환경
- Syscalls: 시스템 콜 핸들러

3. Core Machine (sp1-core-machine)

역할: 증명 생성을 위한 산술화 및 제약 조건 생성

주요 기능:

- AIR (Algebraic Intermediate Representation) 시스템
- 각종 칩(Chip) 구조 관리
- 제약 조건 생성 및 검증
- 다항식 커밋 준비

핵심 구성요소:

- RiscvAir: RISC-V 명령어 AIR
- Chips: CPU, 메모리, 시스템 콜 등의 특화 처리 단위
- StarkMachine: STARK 증명을 위한 머신 인터페이스

4. Prover (sp1-prover)

역할: 다단계 증명 생성 프로세스 조정

증명 생성 단계:

1. **Core Proof**: 개별 샤드에 대한 STARK 증명 생성
2. **Compression**: 여러 샤드 증명을 하나로 압축
3. **Shrinkage**: 증명 크기 최적화
4. **Wrapping**: SNARK-friendly 필드로 변환
5. **Final Wrapping**: Groth16/PLONK 증명으로 최종 변환

주요 구성요소:

- CoreProver: 기본 STARK 증명 생성
- CompressProver: 증명 압축
- ShrinkProver: 증명 축소
- WrapProver: SNARK 래핑

5. Recursion (sp1-recursion-*)

역할: 재귀적 증명 압축 및 합성 시스템

하위 crate들:

- **core**: 재귀적 증명의 핵심 로직
- **compiler**: 재귀 회로 컴파일러
- **circuit**: 재귀 회로 구현
- **gnark-ffi**: Gnark와의 FFI 인터페이스
- **gnark-cli**: Gnark CLI 도구 인터페이스

6. CLI (sp1-cli)

역할: 명령줄 인터페이스 제공

주요 명령어:

- `cargo prove build` : 프로그램 빌드
- `cargo prove trace` : 실행 추적 생성
- `cargo prove prove` : 증명 생성
- `cargo prove verify` : 증명 검증

7. CUDA (sp1-cuda)

역할: GPU 가속화를 통한 성능 최적화

주요 기능:

- CUDA 커널 구현
- GPU 메모리 관리
- 병렬 연산 최적화
- CPU 폴백 지원

8. Verifier (sp1-verifier)

역할: 생성된 증명의 검증

지원 증명 타입:

- Groth16 증명 검증
- PLONK 증명 검증
- no-std 환경 지원
- BN254 곡선 기반 검증

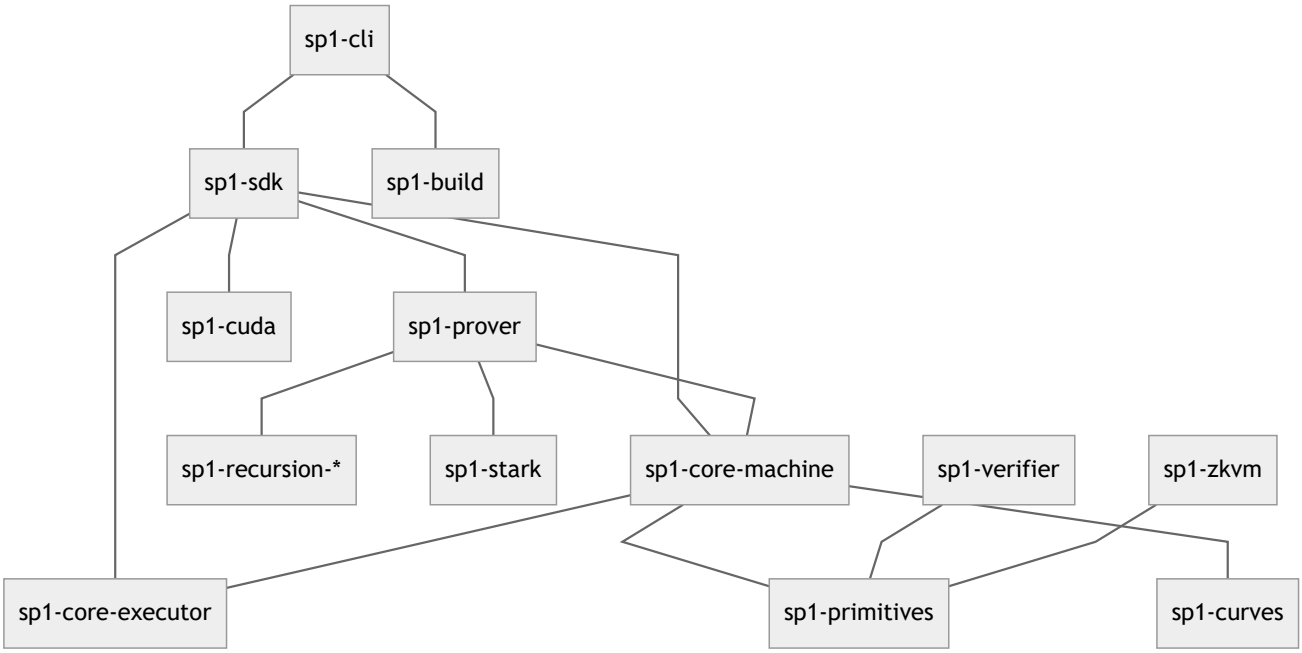
9. zkVM (sp1-zkvm)

역할: zkVM 런타임 환경 제공

구성요소:

- **entrypoint**: 프로그램 진입점 매크로
- **lib**: zkVM 표준 라이브러리

의존성 관계



빌드 시스템 및 도구

Build System (sp1-build)

- Rust 프로그램을 RISC-V ELF로 컴파일
- 크로스 컴파일 도구체인 관리
- 빌드 스크립트 및 매크로 제공

Helper Utilities (sp1-helper)

- 공통 유틸리티 함수
- 로깅 및 디버깅 도구
- 설정 관리

Performance Tools (sp1-perf)

- 성능 측정 및 벤치마킹
- 프로파일링 도구
- 성능 회귀 테스트

외부 의존성

Plonky3 Framework

SP1은 Plonky3 툴킷을 기반으로 구축되어 있습니다:

- p3-field: 유한체 연산
- p3-air: AIR 시스템
- p3-uni-stark: STARK 증명 시스템
- p3-fri: FRI 다항식 커밋

암호학 라이브러리

- BN254 타원곡선 (Groth16/PLONK용)
- Baby Bear 필드 (STARK용)
- Poseidon2 해시 함수
- Keccak 해시 함수

버전 관리

- **현재 버전**: 5.2.1
- **MSRV**: Rust 1.79
- **워크스페이스 기반**: 모든 crate가 동일한 버전 사용
- **의미적 버전 관리**: 호환성을 고려한 버전 업데이트

개발 환경 설정

필수 도구

- Rust 툴체인 (1.79+)
- LLVM (RISC-V 타겟 지원)
- CUDA 툴킷 (GPU 가속화 사용 시)

빌드 명령어

```
# 전체 워크스페이스 빌드
cargo build --release

# 특정 crate 빌드
cargo build -p sp1-sdk --release

# 테스트 실행
cargo test --workspace
```

확장성 및 모듈성

SP1의 아키텍처는 다음과 같은 확장성을 제공합니다:

1. **새로운 Precompile 추가**: 새로운 암호학 연산을 위한 특화 칩 추가 가능
2. **다른 증명 시스템 지원**: 새로운 SNARK 시스템 통합 가능
3. **성능 최적화**: 새로운 하드웨어 가속화 백엔드 추가 가능

4. **언어 지원 확장**: LLVM 기반의 다른 언어 지원 가능

이러한 모듈화된 구조는 SP1이 지속적으로 발전하고 새로운 요구사항에 적응할 수 있게 해줍니다.